



Neural Networks & Deep Learning

2.Mathematical Building Blocks (Gradient based optimization)

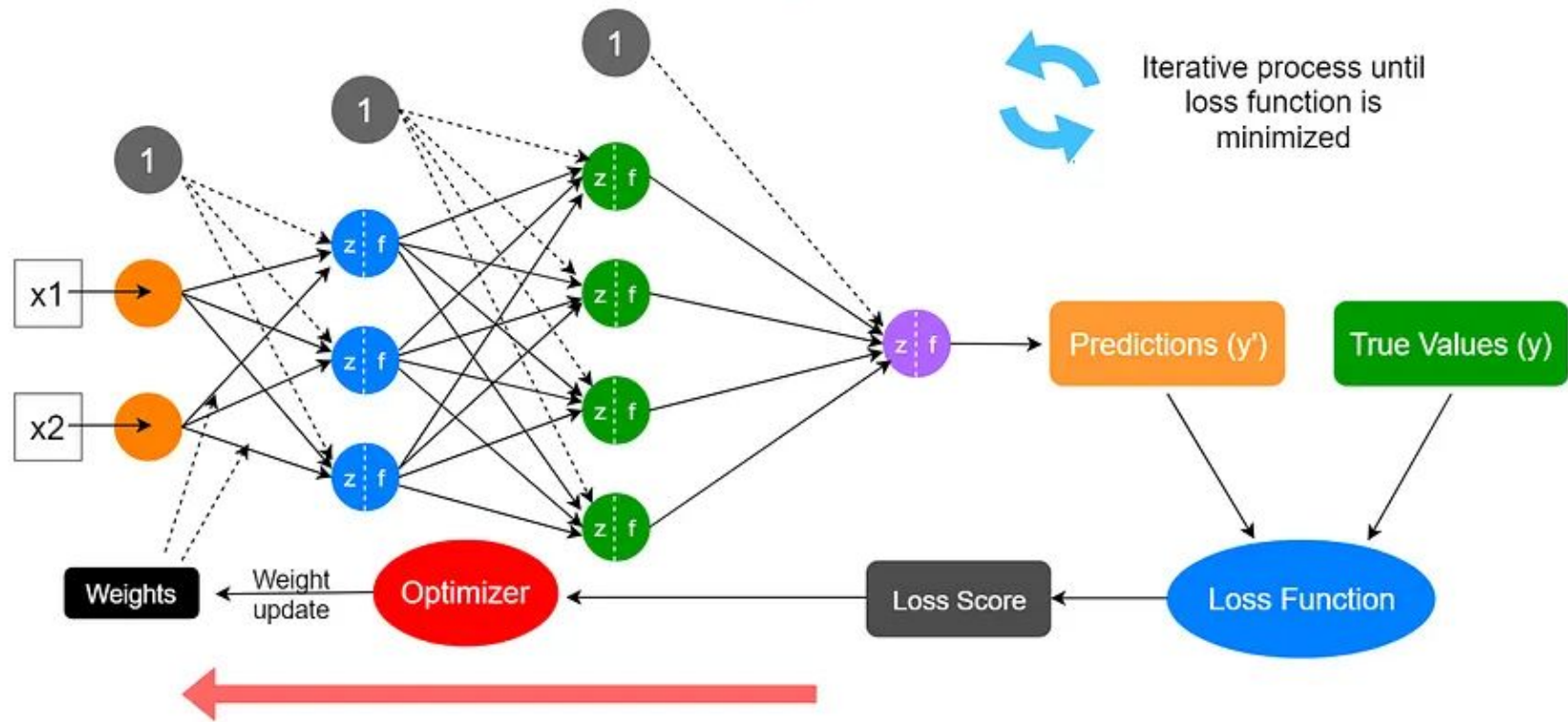
Thenuwan Jayasinghe

19 January 2026

How a Neural Network 'Learns'?



Forward Propagation



Backward Propagation

The training loop works as follows:

1. Draw a batch of training samples x and corresponding targets y_{true} .
2. Run the model on x (a step called the forward pass) to obtain predictions y_{pred} .
3. Compute the loss of the model on the batch, a measure of the mismatch between y_{pred} and y_{true} .
4. Update all weights of the model in a way that slightly reduces the loss on this batch.

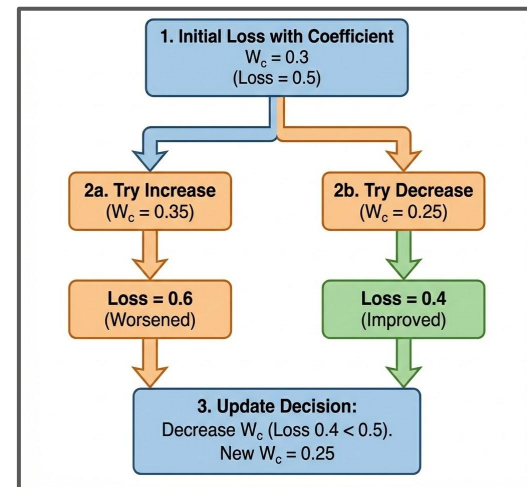
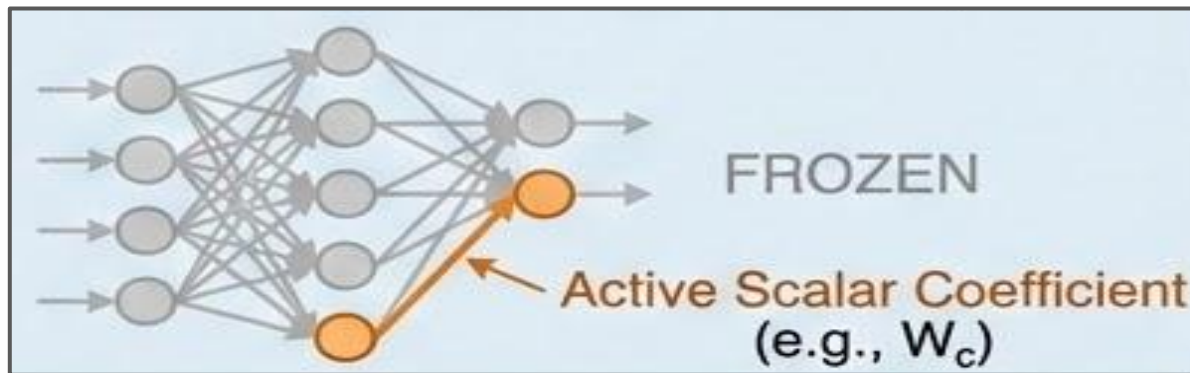
This process gets repeated, and eventually, you will end up with a model that has a very low loss on the training data.

The training loop works as follows:

The difficult part of **Step 04**. How can you compute:

- Whether the coefficient should be **increased** or **decreased**?
- And by **how much**?

The naive approach



The naive approach would be **inefficient** because you would need to compute two forward passes for every individual coefficient (could be billions!)

The naive approach

There is a **better** approach called **'gradient descent'**



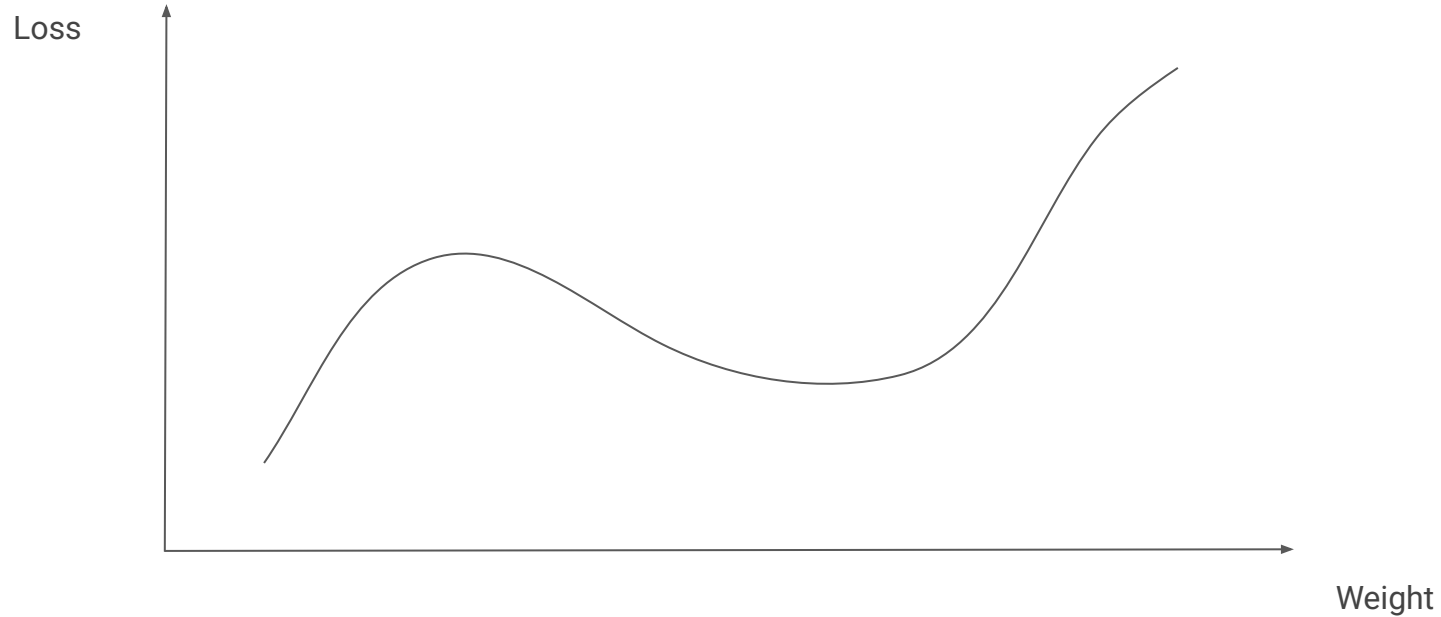
Gradient Descent



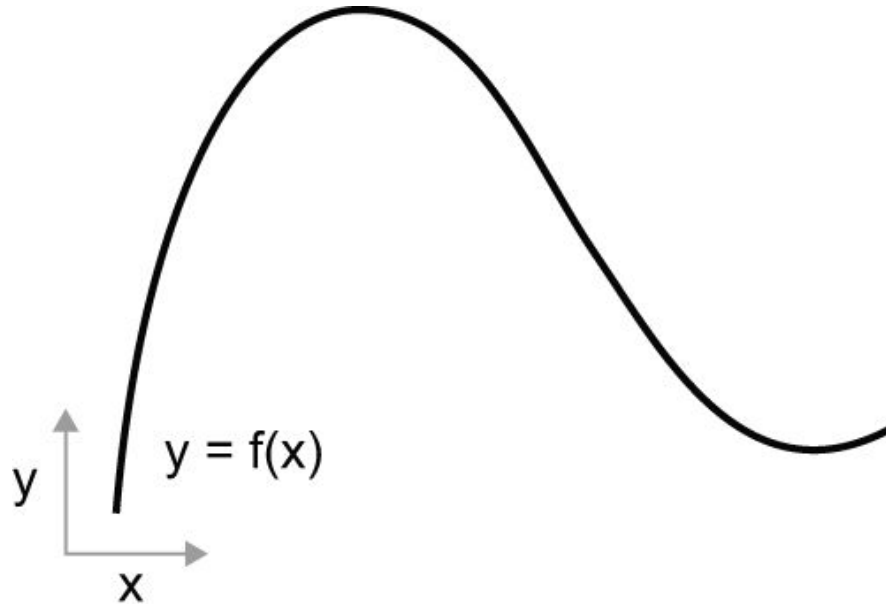
What is a "Differentiable" Function?

- **Smooth & Continuous:** A differentiable function is one that doesn't have sudden jumps or breaks. It's like a smooth slide rather than a set of stairs.
- **Predictable Changes:** If you make a tiny change to the input, the output changes by a tiny, predictable amount.
 - Example: If $z = x + y$, changing y slightly results in a small, known change in z .
- **The "Chain" Effect:** If you stack many smooth functions together (which is exactly what a Neural Network does), the entire model remains smooth and predictable.
- **The Power of the Gradient:** Because the model is differentiable, we can use a mathematical tool called the Gradient to calculate exactly how to "wiggle" our model's weights to make the error (Loss) go down.

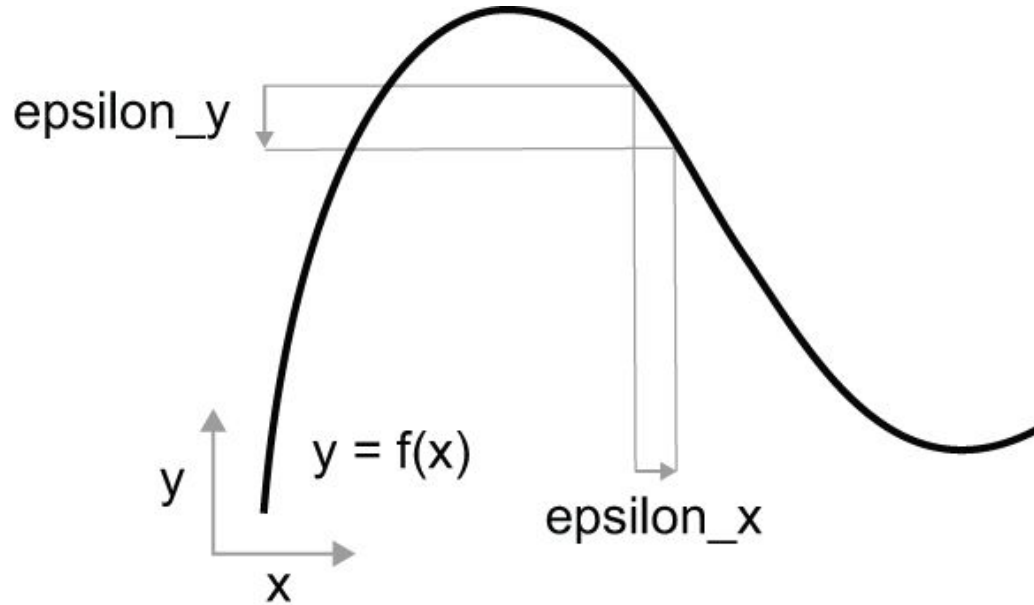
What is a "Differentiable" Function?



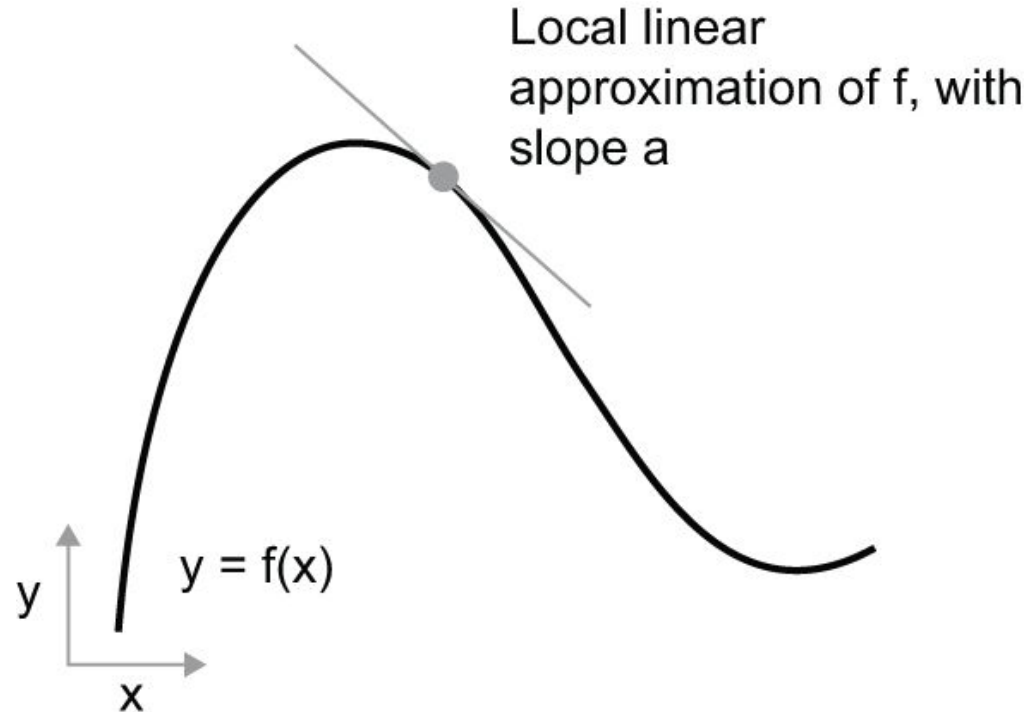
What is a Derivative?



What is a Derivative?



What is a Derivative?

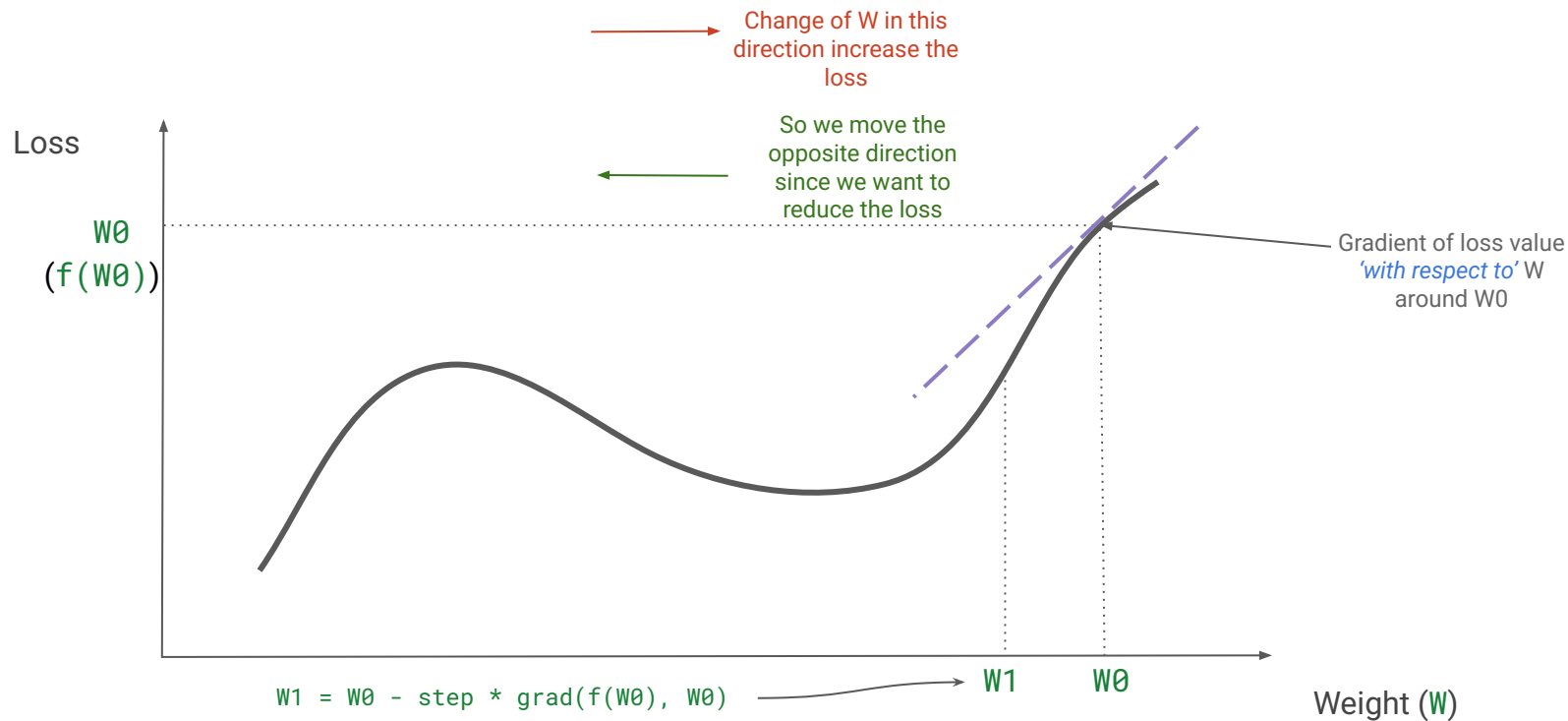


What is a Derivative?

<https://www.derivative-calculator.net/>

<https://www.derivativecalc.com/>

Updating weights

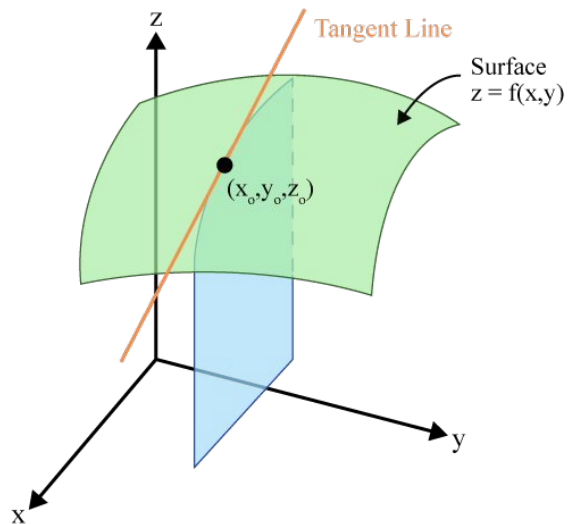


Updating weights

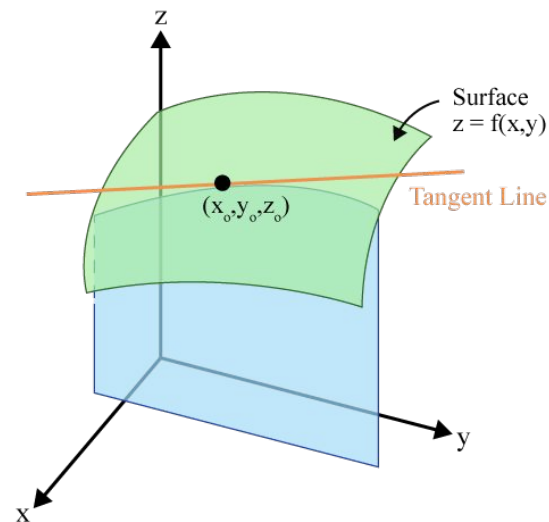
- The derivative of a tensor operation is called a gradient
- The gradient of a tensor function represents the 'curvature' of the multidimensional surface described by the function
- It characterizes how the output of the function varies when its input parameters vary
- You can reduce the loss value by moving the (updating) the weight (W_0) in the opposite direction from the gradient. The new (updated) weight (W_1) value would be

$$W_1 = W_0 - \text{step} * \text{grad}(f(W_0), W_0)$$

More on *'with respect to'...*



Slope of the surface in the x -direction



Slope of the surface in the y -direction

Stochastic Gradient Descent

Why can't we just solve for the minimum?

- For a differentiable function, we can theoretically find the minimum by finding where the derivative is 0
- The Problem: In a neural network, this means solving a polynomial equation for every single weight (coefficient) in the model.
- The Reality: While you could solve this for 2 or 3 variables, real neural networks have thousands or even billions of parameters.
- Solving this analytically is intractable (impossible in practice). We need a way to find the minimum step-by-step.

Stochastic Gradient Descent

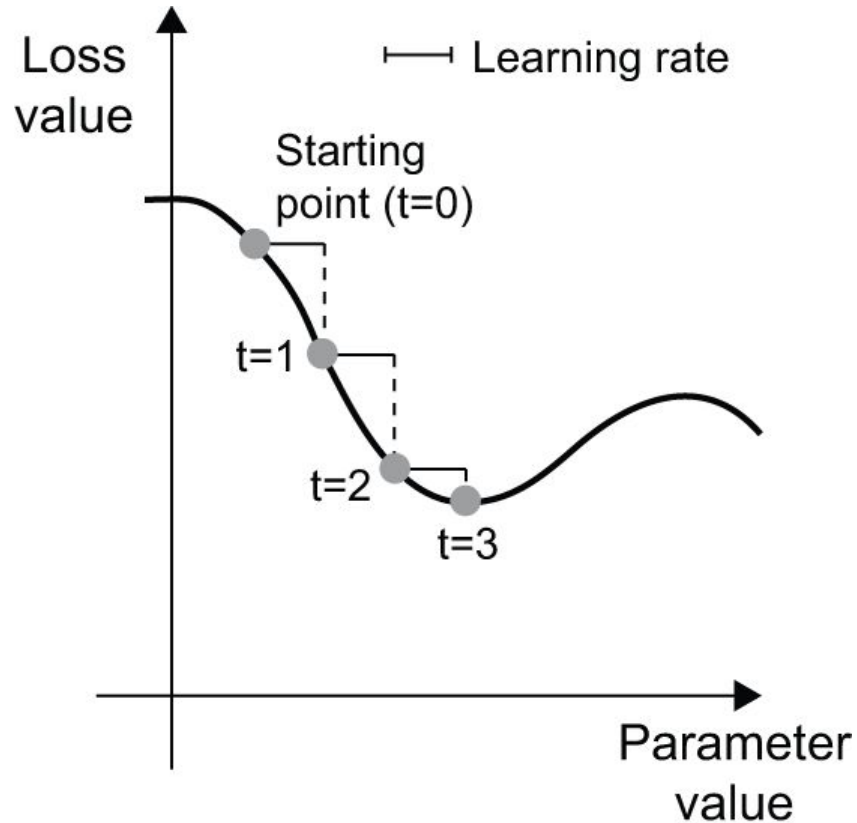
Instead of solving everything at once, we modify parameters "little by little" using random batches of data.

The 4-Step Mini-Batch SGD Algorithm

- Draw a Batch: Get a random set of training samples (x) and their targets (y_{true})
- Forward Pass: Run the model on x to get predictions (y_{pred})
- Compute Loss: Calculate the mismatch between the predictions and the actual targets.
- Backward Pass & Update: Compute the gradient of the loss with respect to the model's parameters.
 - Move parameters slightly in the opposite direction of the gradient

(e.g., $W -= \text{learning_rate} \times \text{gradient}$)

Stochastic Gradient Descent

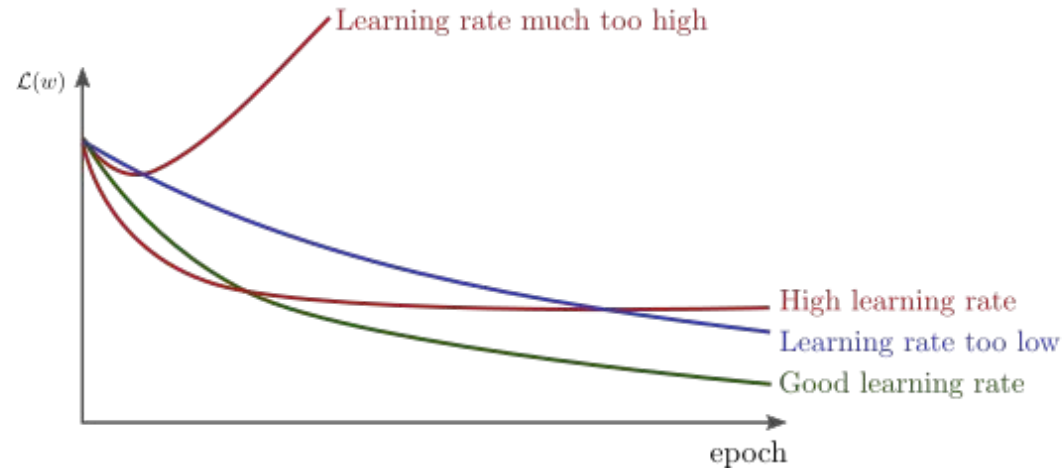
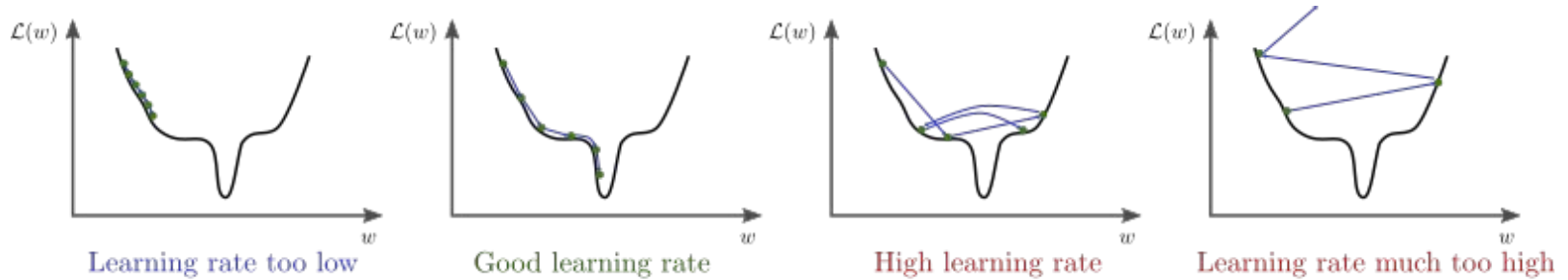


Stochastic Gradient Descent

Understanding the Terminology

- **Stochastic:** A scientific synonym for "random". It refers to the fact that each batch of data is drawn at random
- **Mini-batch:** Using a small, reasonable number of samples (the "efficient compromise") rather than just one sample or the entire dataset at once.
- **The Learning Rate:** A scalar factor that acts as the "speed" of the descent
 - Too Small: The descent takes too long and can get stuck in a "local minimum".
 - Too Large: Updates might jump to completely random locations on the curve

Stochastic Gradient Descent

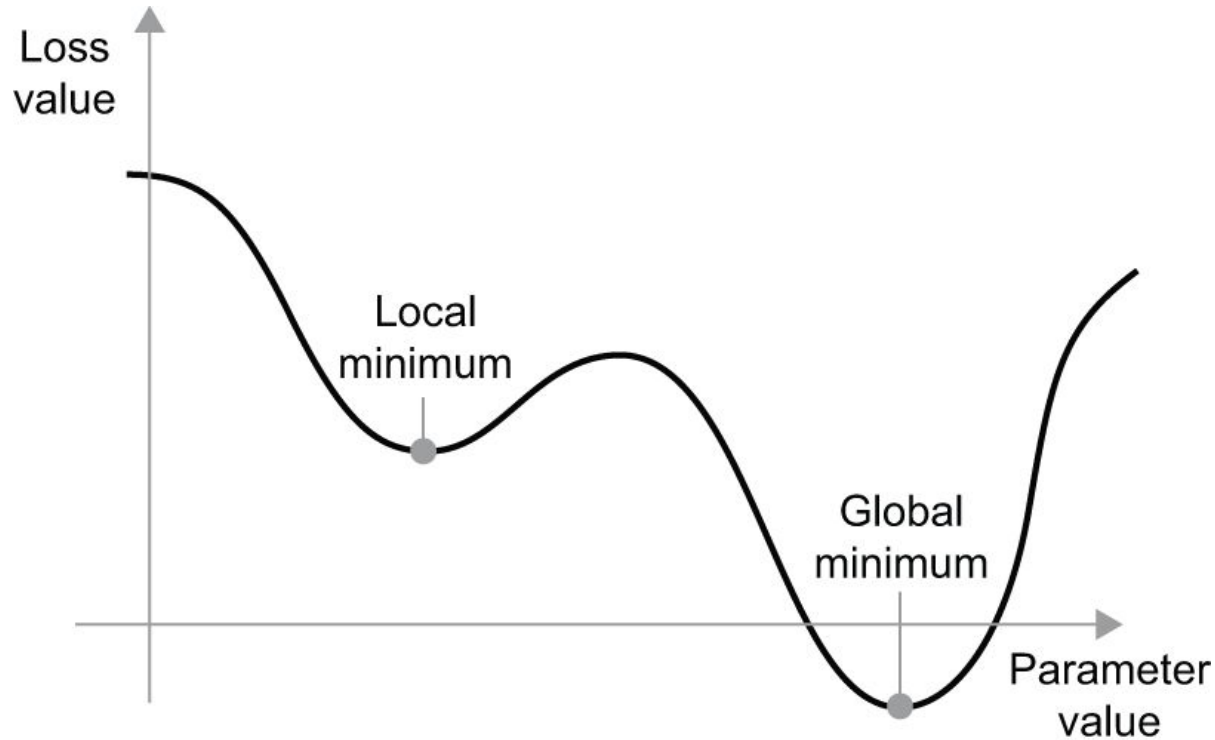


Stochastic Gradient Descent

Challenges - Local vs. Global Minima

- Global Minimum: The absolute lowest point of the loss function (our ultimate goal).
- Local Minimum: A point where moving in any direction increases the loss, but it isn't the absolute lowest point.
- The Danger: With a small learning rate, SGD can get "trapped" in a local minimum and never find the best possible solution.

Stochastic Gradient Descent



Stochastic Gradient Descent

Momentum

- The Concept: Inspired by physics, momentum helps the optimization process act like a small ball rolling down a curve.
- How it Works: The update is based on the current slope (acceleration) AND the current velocity (past acceleration).
- The Benefit: If the ball has enough momentum, it can roll right through a small "ravine" (local minimum) and continue toward the global minimum.
- Popular Optimizers: Many modern optimizers use momentum, including SGD with Momentum, RMSprop, and Adagrad.

How do we compute gradients?

- We know we need the gradient of the loss with respect to our weights.
- How do we compute gradients for incredibly complex expressions with millions of interlocking parts?
- The Solution: The Backpropagation algorithm.
- Backpropagation is simply the application of a mathematical identity called the Chain Rule to a network.

Chain Rule



The Chain Rule - The Concept

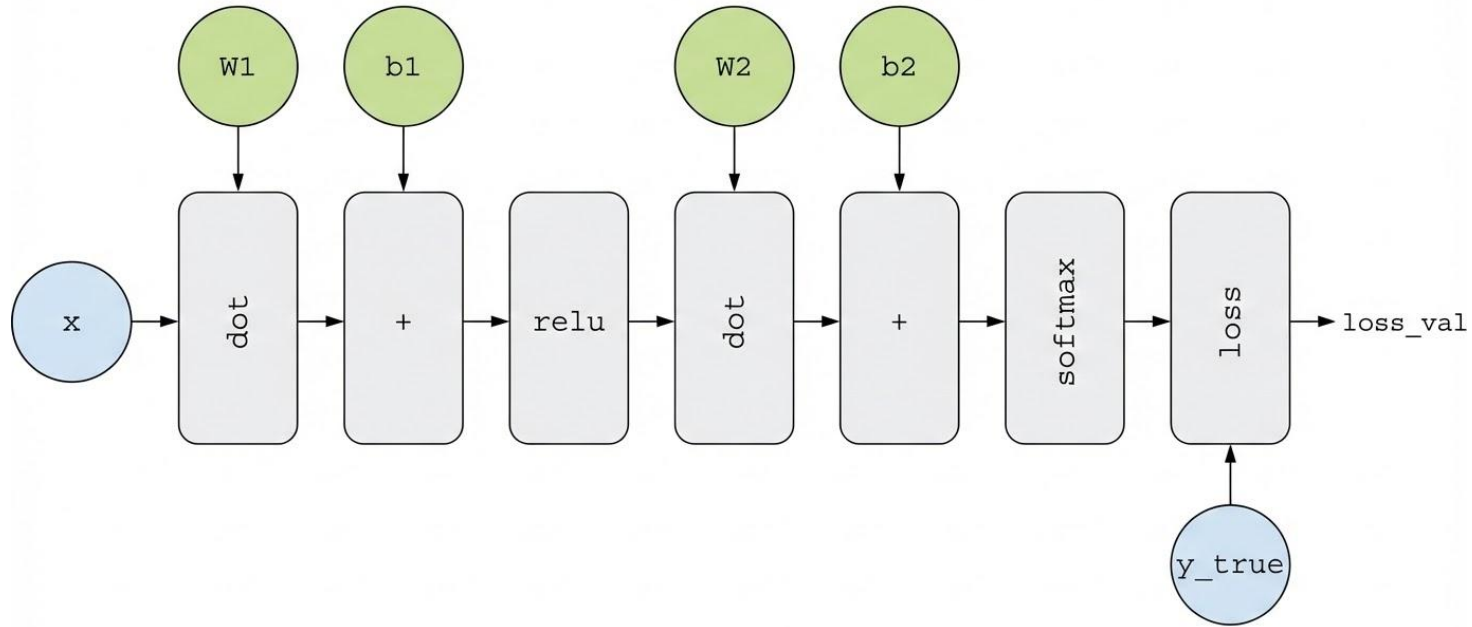
- A neural network is just a chain of simple, "atomic" operations (like addition, ReLU, or tensor products)
- If you know the derivative of the simple parts, you can compute the derivative of the complex whole.
- The chain rule is a fundamental calculus technique for finding the derivative of composite functions.
- The Math:
 - If $h(x) = f(g(x))$ then the derivative of h with respect to x denoted as $h'(x)$ is given by:
 - $h'(x) = f'(g(x)) \cdot g'(x)$
- Why it's a "Chain": As you add more functions, you simply keep multiplying the gradients along the path

Computation Graph

Visualizing with Computation Graphs

- A computation graph is a Directed Acyclic Graph (DAG) of tensor operations.
- It's easier for a computer program to calculate derivatives if the math is represented as an explicit data structure
- Automatic Differentiation: This graph structure allows modern frameworks (PyTorch, TensorFlow) to retrieve gradients for any complex model without you having to do the math by hand.

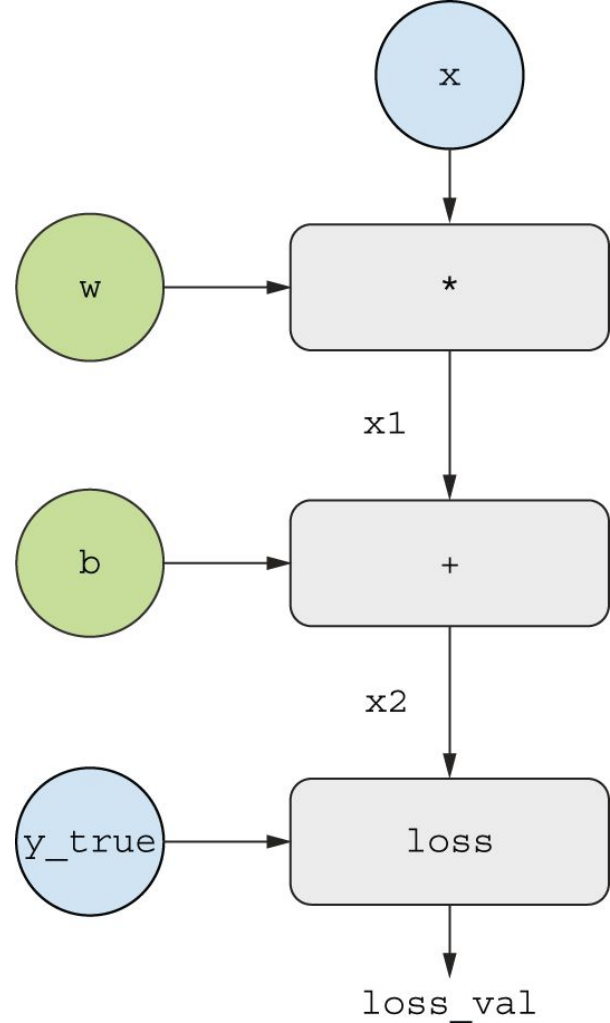
Computation Graph



Computation Graph

Example

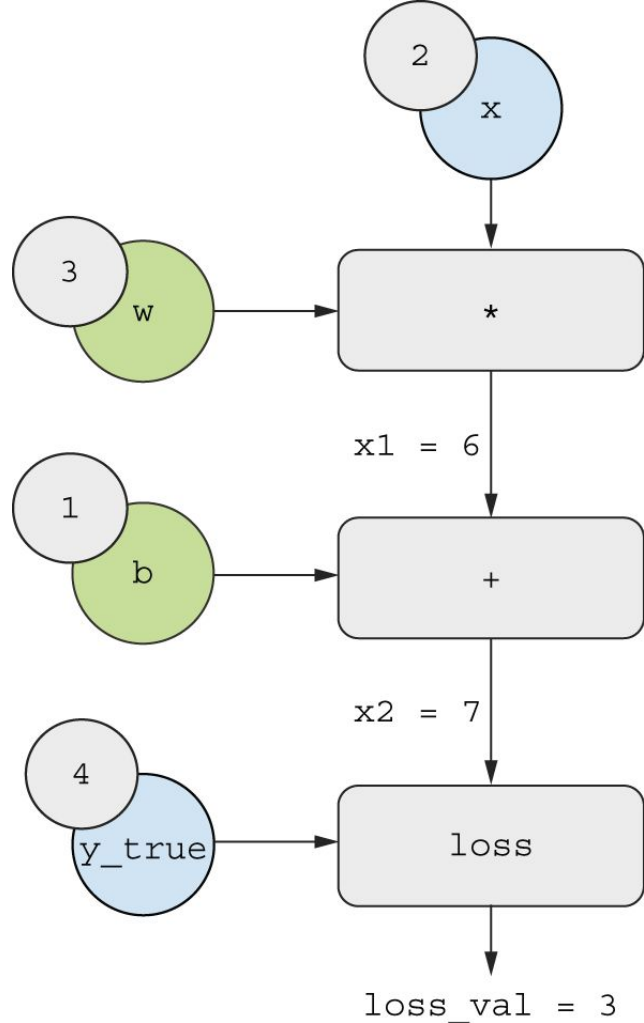
- One linear layer
- All variables are scalar
 - Scalar variables : w, b
 - Scalar Inputs : x
 - Output : y
 - Loss : $\text{abs}(y_{\text{true}} - y)$
- We want to calculate
 - $\text{grad}(\text{loss_val}, b)$
 - $\text{grad}(\text{loss_val}, w)$



Computation Graph

Example

- One linear layer
- All variables are scalar
 - Scalar variables : $w = 3, b = 1$
 - Scalar Inputs : $x = 2$
 - Output : y
 - Loss : $\text{abs}(y_{\text{true}} - y) = 3$
- We want to calculate
 - $\text{grad}(\text{loss_val}, b)?$
 - $\text{grad}(\text{loss_val}, w)?$



Questions & Answers



www.boffin.edu.lk

thenuwanji@boffin.lk

+94716296777

[LinkedIn](#)